

INDIAN INSTITUTE OF TECHNOLOGY DELHI

Department of Electrical Engineering

ELL305: Computer Architecture — Term Paper

Modern Processor Design Beyond Monolithic Scaling

Chiplet Architectures, Cache Hierarchies, and 3D-Stacked Cache — A Case Study of AMD Zen Processors

Initial Full-Paper Working Draft (draft-initial)

Submitted by

Ishan Krishna Agrawal	2024MT60860	mt6240860@maths.iitd.ac.in
Vaidik Goyal	2024EE11030	ee1241030@ee.iitd.ac.in
Moon Saraswat	2024EE11163	ee1241163@ee.iitd.ac.in

28 October 2026

Copyright and Intellectual Property Rights Assignment

All intellectual property rights in the original content of this report belong to the Indian Institute of Technology Delhi, including its Department of Electrical Engineering.

Intellectual property rights in material cited or otherwise attributed to external sources remain with the respective authors, publishers, or other rights holders.

The Authors
28 October 2026

Acknowledgements

The authors acknowledge Prof. Sumantra Dutta Roy as the faculty advisor for this term-paper project.

The authors also acknowledge the Department of Electrical Engineering and the Indian Institute of Technology Delhi for the academic environment, facilities, and institutional resources supporting this work.

This paper is prepared collectively by Ishan Krishna Agrawal, Vaidik Goyal, and Moon Saraswat.

Abstract

For several decades, processor performance improved largely by integrating more transistors on a single, monolithic die. That approach is now constrained by die size limits, manufacturing yield, the uneven scaling of logic, SRAM and analog circuits, and the power and thermal budget of the package. This paper studies two linked architectural responses to these constraints: chiplet-based processor organisation, which partitions a processor into separately manufactured dies, and three-dimensional stacking of cache, which adds last-level-cache capacity without enlarging the compute die. AMD Zen processors serve as the principal case study, tracing the progression from the compute-die and I/O-die partitioning of Zen 2, through the unified eight-core L3 organisation of Zen 3, to the hybrid-bonded 3D V-Cache products. For each technique the paper asks which bottleneck it addresses, what benefit it provides, and what new cost or trade-off it introduces. The analysis is grounded in the memory-hierarchy and multiprocessor foundations of the course textbook and evaluates published research and benchmark evidence rather than new hardware measurements. **[TODO:** Revise the abstract once Parts V and VI contain completed analysis.]

Keywords: processor architecture, chiplets, memory hierarchy, last-level cache, 3D-stacked cache, hybrid bonding, AMD Zen, ELL305.

Contents

List of Tables **8**

List of Figures **9**

Glossary **10**

I Introduction and Research Framing 12

1 Introduction 12

1.1 Motivation: why monolithic scaling is becoming difficult 12
1.2 Two linked scaling problems 13
1.3 Research questions and objectives 13
1.4 Scope 14
1.5 Methodology 15
1.6 Contributions 15
1.7 Organisation of the paper 15

II Architectural Foundations 16

2 Processor Organisation and Performance Metrics 16

2.1 The performance equation 16
2.2 Baseline processor organisation 16

3 Pipelining, Power and Thermal Limits 17

3.1 Pipelining and its limits 17
3.2 Dynamic and leakage power 17
3.3 Temperature 17

4 The Memory Hierarchy 17

4.1 Why a hierarchy is needed 17
4.2 Average memory access time 18
4.3 Classifying cache misses 19

4.4 Cache organisation and policies 19
4.5 Beyond SRAM: alternative LLC technologies 19

5 Multicore Processors, Coherence and Interconnects 19
5.1 From one core to many 19
5.2 Cache coherence 20
5.3 On-chip and on-package interconnection networks 20

III Chiplet-Based Processor Architecture 21

6 Why Large Monolithic Dies Become Costly 21
6.1 The reticle limit and die size 21
6.2 Manufacturing yield 21
6.3 Uneven scaling of logic, SRAM and analog circuits 22

7 Compute Dies, I/O Dies and Core Complexes 22

8 Infinity Fabric and Die-to-Die Communication 23
8.1 Latency, bandwidth and interconnect power 24
8.2 Scalability 24

9 Yield, Process-Node Specialisation and Cost 24

10 Summary of Part III 24

IV 3D-Stacked Cache and AMD 3D V-Cache 25

11 Why Cache Capacity Matters 25

12 Why Expanding Cache in Two Dimensions Became Difficult 25
12.1 Area and yield 25
12.2 Wire length and access latency 26
12.3 Power 26
12.4 Summary of the planar limits 26

13 Side-by-Side Versus Vertically Stacked Cache Dies **26**

13.1 Placing the cache die beside the compute die 26

13.2 Stacking the cache die on the compute die 27

14 Stacked-Die Integration and Hybrid Bonding **28**

14.1 The first-generation 3D V-Cache stack 28

14.2 Hybrid bonding versus micro-bumps 29

15 Cache Access, Latency, Bandwidth and Hierarchy **29**

16 Thermal Management, Power Delivery and Manufacturing Complexity **30**

17 Workloads That Benefit From a Larger LLC **30**

18 Summary of Part IV **31**

V AMD Case Study and Quantitative Analysis **32**

19 From Zen 2 Chiplets to Zen 3 and 3D V-Cache **32**

20 Comparing Architectural Configurations **33**

21 Method for Analysing Published Evidence **33**

22 Published Benchmarks and Performance Measurements **34**

23 Cache-Sensitive Versus Cache-Insensitive Workloads **34**

24 Separating Cache Effects From Other Factors **35**

25 Summary of Part V **35**

VI Comparative Synthesis, Limitations and Future Directions **36**

26 Monolithic, Chiplet-Based and Vertically Stacked Designs Compared **36**

27 Interactions Between Chiplet Organisation and Cache Design **37**

28 Workload-Dependent Trade-Offs and Remaining Bottlenecks **37**

29 Future Directions in Heterogeneous Integration and Advanced Packaging **37**

30 Limitations of This Study **37**

31 Conclusions **38**

32 Open Research Questions **38**

References **39**

A Plagiarism Report **40**

B Reading Indices **41**

C Yield and Cost-per-Good-Die Model **42**

C.1 From Poisson to Murphy yield 42

C.2 Cost per good die 42

C.3 Splitting a design into chiplets 43

Index **44**

List of Tables

1 Illustrative Murphy-model yield for $D_0 = 0.2$ defects/cm^2 and $Y_{\text{wafer}} = 1$ 22

2 Qualitative comparison of side-by-side and vertically stacked cache dies 28

3 L3 organisation per compute die across the generations studied (AMD-published figures) 33

4 AMD-reported performance figures relevant to this study (vendor claims, not independent measurements) 34

5 Qualitative comparison of the three design approaches (provisional) 36

List of Figures

1 Research framework: two linked scaling problems, the AMD responses studied in this paper, and the evaluation question applied to each. 14

2 A multi-level cache hierarchy with the quantities used in the recursive AMAT expression (equation (3)). Capacity grows and speed falls moving down the hierarchy. 18

3 Murphy yield model (equation (4), $Y_{\text{wafer}} = 1$) for three illustrative defect densities. The marked points reproduce the worked example in section 6.2. Illustrative calculation, not measured processor data. 22

4 Schematic contrast between a monolithic processor die and a chiplet package with compute dies (CCDs) and an I/O die (IOD). Illustrative only; not to scale and not an actual AMD floorplan. 23

5 Two ways to add cache capacity to a compute die. Illustrative arrangements only; not actual AMD package layouts and not to scale. 27

6 Schematic progression of L3 organisation per compute die from Zen 2 to Zen 3 and Zen 3 with 3D V-Cache, based on AMD's descriptions [3], [4], [5]. Logical organisation only; not a floorplan. 32

7 Planned evidence-analysis procedure for Part V (published research and benchmarks only; no new hardware measurement). 33

Glossary

3D V-Cache AMD's name for a design that uses three-dimensional die integration to add stacked cache capacity above a processor die.

AMAT (average memory access time) A weighted average of memory-access latencies that accounts for cache hits, cache misses, and the penalties of accessing lower levels of the memory hierarchy.

cache coherence The mechanisms and rules used to maintain a consistent view of shared data when multiple caches can hold copies of the same memory location.

cache hierarchy A set of cache levels organised to exploit locality by balancing capacity, access latency, and implementation cost.

CCD (Core Complex Die) AMD terminology for a compute die containing processor cores and associated cache resources in selected Zen-based processors.

CCX (Core Complex) AMD terminology for a group of cores that share one L3 cache; in Zen 2 a CCD contains two four-core CCXs, and in Zen 3 a CCD contains one eight-core CCX.

chiplet A separately fabricated die designed to operate with one or more other dies as part of a packaged system.

CPI (cycles per instruction) The average number of clock cycles taken per dynamic instruction; its reciprocal is IPC.

defect density The average number of yield-killing manufacturing defects per unit area of silicon, written D_0 in simple yield models.

hybrid bonding A fine-pitch die-stacking technique that forms direct mechanical and electrical connections between aligned die surfaces.

IFOP (Infinity Fabric On-Package) The on-package serial link used by AMD to connect compute dies to the I/O die over short organic-package routes.

Infinity Fabric AMD's interconnect architecture for communication among processor components and, in relevant products, among separate dies.

IOD (I/O Die) AMD terminology for a die that centralises input/output, memory-interface, and supporting interconnect functions in selected chiplet-based Zen processors.

IPC (instructions per cycle) The average number of dynamic instructions completed per clock cycle; the reciprocal of CPI.

LLC (last-level cache) The final cache level checked before a request proceeds to main memory in the cache hierarchy under discussion.

memory hierarchy An organisation of storage levels with different capacities, costs, and access times, arranged to exploit temporal and spatial locality.

micro-bump A small solder-based bump used to connect stacked or adjacent dies; its pitch limits the achievable interconnect density compared with hybrid bonding.

monolithic die A single semiconductor die on which the principal processor resources under discussion are integrated rather than partitioned across multiple dies.

reticle limit The maximum die area that a lithography system can pattern in a single exposure field, which bounds the size of a monolithic die.

SRAM (static random-access memory) A memory built from bistable cells that retain data while powered without refresh; the usual technology for on-chip caches.

TSV (through-silicon via) A vertical electrical connection passing through a silicon die to support communication in a stacked-die assembly.

yield The fraction of manufactured dies that function correctly; larger dies generally yield worse for a given defect density.

Part I

Introduction and Research Framing

Target length: about 10 pages. Draft status: opening argument, research questions, scope and method drafted; contributions provisional.

1 Introduction

1.1 Motivation: why monolithic scaling is becoming difficult

For most of the history of the microprocessor, a new product generation could be built by placing more transistors on a single piece of silicon. The course textbook summarises this trend as Moore's law: the number of transistors on a chip has roughly doubled every one to two years since the 1960s [1, sec. 12.1.1]. Designers turned those transistors into deeper pipelines, wider issue, larger caches and, from the mid-2000s, more cores per chip [1, ch. 12].

Three pressures now make it harder to keep growing a single monolithic die:

- **Die size and yield.** A lithography system can only pattern a die up to the reticle limit, and the probability that a die contains a fatal defect rises with its area. AMD states that the 64-core server configuration of its second-generation EPYC processors would lie well beyond the maximum reticle size if built as one chip [2]. Section 6.2 quantifies the yield effect with a simple model.
- **Uneven scaling of circuit types.** Logic, SRAM (static random-access memory) and analog/I-O circuits do not shrink equally when moving to a new process node. Naffziger *et al.* note that analog circuitry shrinks far less than digital logic and SRAM, so memory interfaces and SerDes gain little from an advanced node [2]. More recent AMD material indicates that SRAM itself now scales more slowly than logic (section 11).
- **Power and temperature.** Dynamic power grows with switched capacitance, the square of the supply voltage and clock frequency, while leakage power is consumed even when circuits are idle and itself rises with temperature [1, sec. 10.10]. Every additional square millimetre of active silicon must be powered and cooled within a fixed package budget.

These pressures do not stop progress, but they change its form. Instead of one ever-larger die, designers increasingly assemble a processor from several dies inside one package, and increasingly use the vertical dimension as well as the horizontal one. This paper studies that shift through two concrete techniques and one principal case study.

1.2 Two linked scaling problems

The paper is organised around two related problems (figure 1).

Problem 1: scaling processor resources beyond a single die. If a product needs more cores, memory channels and I/O than one economical die can hold, the processor can be partitioned into chiplets. AMD's Zen 2-based products separate eight-core compute dies (CCDs) built on a 7 nm process from an I/O die (IOD) built on a mature 12 nm process [2], [3].

Problem 2: scaling cache capacity without spending planar die area. A larger LLC (last-level cache) can reduce costly trips to main memory, but on a planar die every extra megabyte of SRAM enlarges the die, lowers yield and lengthens wires. AMD's 3D V-Cache instead stacks an additional SRAM die on top of the compute die, expanding the Zen 3 L3 from 32 MB to 96 MB per CCD [4], [5].

The problems are linked: the chiplet organisation makes the CCD a small, self-contained compute die with its own L3, and that die is the natural host for a stacked cache. The paper therefore follows the argument in four steps:

1. Monolithic CPUs face die-size, yield and scaling limits; chiplets address compute scaling [2].
2. Chiplets alone leave the cost of large on-die caches unsolved: AMD's ISSCC 2022 presentation names SRAM scaling and cost as the barriers to large on-die caches, in cost, product flexibility and latency, and argues that 2.5D chiplets address only part of the problem [4, slides 6–7].
3. Vertical stacking addresses those cache-specific limits [4], [6].
4. Evaluation evidence shows where the stacked cache improves performance, and which thermal, packaging and cost trade-offs it introduces (Part V).

1.3 Research questions and objectives

The central research question is:

> *How do chiplet-based organisation and 3D-stacked cache help modern processors address limits in performance, scalability, memory latency, power and die-level integration, and what new trade-offs do they introduce?*

It is divided into four sub-questions:

RQ1 Which architectural and manufacturing limits make a single large die an increasingly costly way to scale a processor?

RQ2 How does partitioning a processor into compute and I/O dies change memory latency, bandwidth, power and product flexibility?

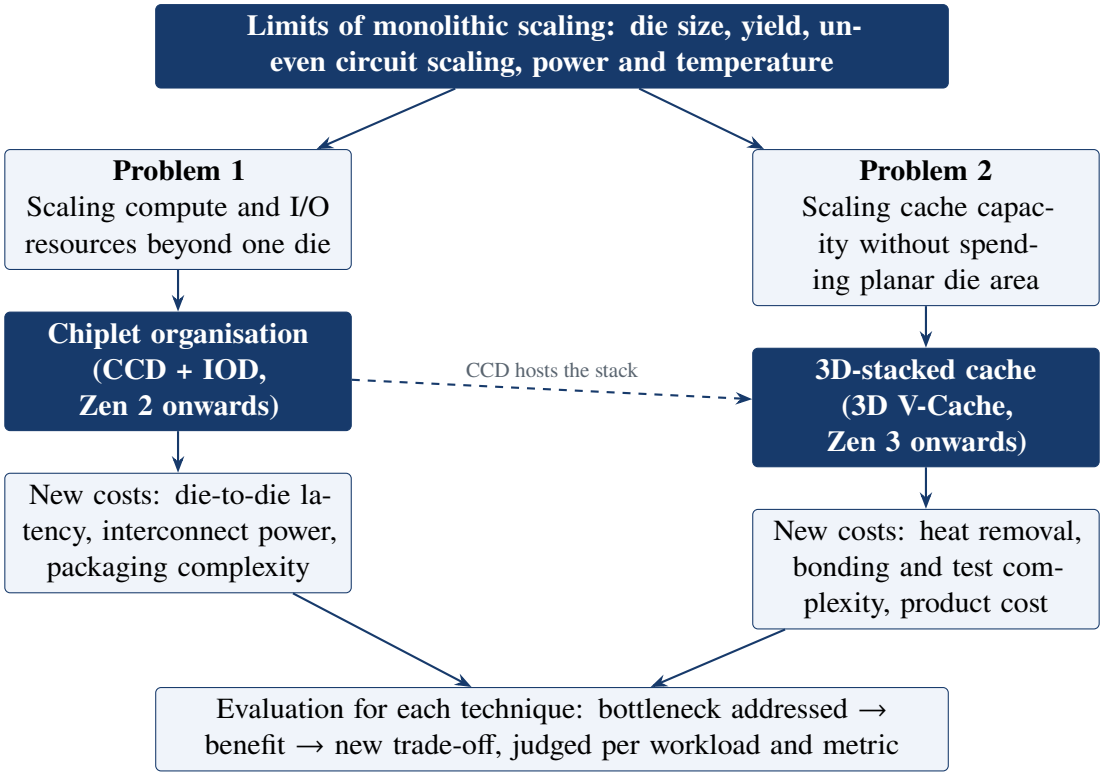


Figure 1: Research framework: two linked scaling problems, the AMD responses studied in this paper, and the evaluation question applied to each.

RQ3 Why does last-level-cache capacity matter, and what does vertical stacking change compared with expanding the cache in the plane of the die?

RQ4 For which workloads does a larger stacked L3 produce measurable benefit, and how can that benefit be separated from differences in frequency, core count and memory configuration?

For every technique the paper applies the same three-part test: *which bottleneck does it address, what benefit does it provide, and what new bottleneck, cost or trade-off does it create?*

1.4 Scope

AMD Zen processors are the principal case study because AMD has published unusually detailed technical descriptions of both its chiplet organisation and its stacked cache [2], [3], [5], [6]. They are, however, a case study and not the whole subject: the architectural principles apply to chiplet and die-stacked designs in general.

The following are deliberately outside scope:

- fabrication chemistry and process details beyond what is needed to explain an architectural decision;
- reverse engineering of undisclosed Zen internals;
- a catalogue of product specifications;

- vendor contests. Comparisons are always tied to a stated workload, metric and time period, and the paper makes no general claim that one vendor "beats" another.

1.5 Methodology

The study combines three kinds of evidence:

1. **Textbook foundations.** Chapters 9–12 of Sarangi's *Basic Computer Architecture* supply the models used throughout: the performance equation, pipelining limits, power and temperature, the memory hierarchy and average memory access time, and multicore coherence and interconnects [1].
2. **Primary technical disclosures.** Peer-reviewed conference papers and vendor technical presentations describe what was built and why. Vendor presentations are attributed to the vendor and treated as engineering claims rather than independent measurements.
3. **Published research and benchmarks.** Part V analyses published measurements of chiplet and stacked-cache processors. The paper does not perform new hardware measurements; its quantitative contribution lies in organising, normalising and critically comparing existing results (section 21).

1.6 Contributions

[TODO: Finalise once Parts V and VI are complete. Provisional list:**]**

- a single argument linking chiplet partitioning and 3D-stacked cache as responses to two related scaling limits;
- an explanation of both techniques in terms of the textbook's performance, power and memory-hierarchy models;
- a structured comparison of planar, side-by-side and vertically stacked cache expansion;
- a critical synthesis of published performance evidence for stacked cache, with explicit treatment of confounding factors.

1.7 Organisation of the paper

Part II reviews the architectural foundations. Part III analyses chiplet-based organisation. Part IV examines 3D-stacked cache and AMD 3D V-Cache. Part V presents the AMD case study and quantitative analysis, and Part VI compares the approaches, identifies limitations and outlines future directions. Self-contained derivations longer than about half a page appear as appendices.

Part II

Architectural Foundations

Target length: about 25 pages. Draft status: core models drafted (performance equation, power, AMAT, miss classes); pipelining, cache organisation and coherence chapters need expansion.

This part sets out the models that the rest of the paper uses to judge chiplets and stacked cache. It follows Chapters 9–12 of the course textbook [1]. The textbook supplies the theory; it does not describe modern compute-die/I/O-die partitioning or stacked cache, which later parts support with external sources.

2 Processor Organisation and Performance Metrics

2.1 The performance equation

The textbook defines the performance of a processor only with respect to a given program, as the reciprocal of its execution time [1, sec. 10.9.1]. Execution time τ can be split into three factors:

$$\tau = \frac{\#\text{instructions} \times \text{CPI}}{f}, \qquad P \propto \frac{\text{IPC} \times f}{\#\text{instructions}}, \tag{1}$$

where f is the clock frequency, CPI (cycles per instruction) is the average number of cycles per dynamic instruction, and IPC (instructions per cycle) is its reciprocal [1, sec. 10.9.1].

Equation (1) is the lens for the whole paper. A larger cache does not change the instruction count or, directly, the clock frequency; it acts through IPC by reducing the cycles spent waiting for memory. A chiplet organisation, by contrast, can change all three: it can enable more cores (more throughput), alter memory latency (IPC), and change the frequency and power each die can sustain. When published results compare two processors, they must therefore be checked for differences in f as well as in cache size (section 24).

2.2 Baseline processor organisation

Planned content (not yet written). Summarise the single-cycle and multi-cycle processor organisation of Chapter 9 as the baseline from which pipelined, multicore and partitioned designs depart. Keep to what is needed for later parts (about two pages). Source: [1, ch. 9].

3 Pipelining, Power and Thermal Limits

3.1 Pipelining and its limits

Planned content (not yet written). Pipeline stages, hazards and their effect on CPI; why memory stalls are a principal source of non-ideal CPI; brief note on out-of-order execution hiding some, but not all, memory latency [1, ch. 10]. About three pages.

3.2 Dynamic and leakage power

The textbook models dynamic power, the energy dissipated when circuit nodes switch, as

$$P_{\text{dyn}} \propto \sum_i \alpha_i C_i V^2 f, \tag{2}$$

where α_i is the activity factor and C_i the capacitance of node i, V the supply voltage and f the frequency [1, sec. 10.10.2]. Leakage (static) power arises because transistors do not switch off perfectly; it is consumed all the time regardless of activity, and the textbook notes that it can account for roughly 20–40% of a high-performance processor's power budget [1, sec. 10.10.3].

Both terms matter for this paper:

- **Wires are capacitance.** Long on-die or die-to-die wires increase C in equation (2), so moving data farther costs energy as well as time. This is one reason the distance between cores and cache matters (section 12).
- **Large SRAM arrays leak.** A large cache adds leakage even when most of it is idle, which is part of the cost of simply growing the LLC on a planar die.

3.3 Temperature

Power that is dissipated becomes heat, which must flow from the transistors through the die, a thermal interface material and a heat spreader to the cooler [1, sec. 10.10.1]. Higher temperature raises leakage, which raises power further, and accelerates ageing [1, sec. 10.10.1]. Stacking a die on top of a compute die changes this heat path, which is the main thermal question raised in section 16.

[TODO: Add the ED^2 metric ([1, sec. 10.10.5]) if Part V uses energy-delay comparisons.**]**

4 The Memory Hierarchy

4.1 Why a hierarchy is needed

The textbook frames memory design as a conflict between capacity and speed. Per unit of silicon, DRAM stores far more data than SRAM but is much slower; a memory built entirely of DRAM

would make every load and store wait for main memory, while a memory built entirely of fast on-chip SRAM cannot hold a program's data [1, sec. 11.1.1]. As an illustration, if one third of instructions access a 100-cycle memory, the effective CPI of an otherwise ideal pipeline rises to $1 + \frac{1}{3}(100 - 1) = 34$ [1, sec. 11.1.1].

The way out is *locality*: programs tend to reuse recently used data (temporal locality) and to access data near recently used data (spatial locality) [1, sec. 11.1.2–11.1.6]. A memory hierarchy keeps recently used blocks in small, fast caches close to the core and backs them with progressively larger, slower levels (figure 2).

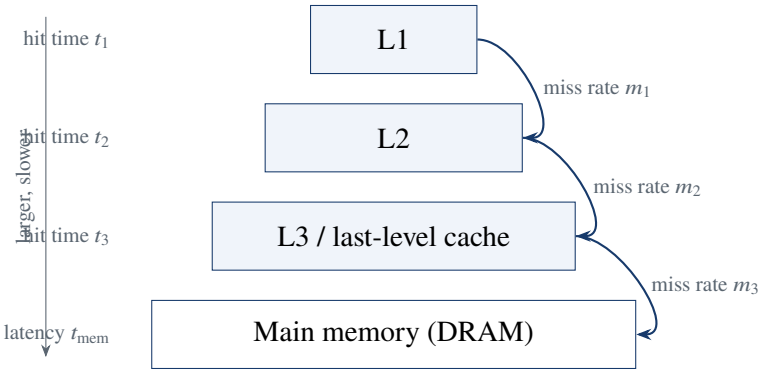


Figure 2: A multi-level cache hierarchy with the quantities used in the recursive AMAT expression (equation (3)). Capacity grows and speed falls moving down the hierarchy.

4.2 Average memory access time

The average memory access time (AMAT (average memory access time)) of a hierarchy is defined recursively [1, sec. 11.3.1]:

$$\begin{aligned} \text{AMAT} &= t_1 + m_1 \times \text{penalty}_1, \\ \text{penalty}_k &= t_{k+1} + m_{k+1} \times \text{penalty}_{k+1}, \end{aligned} \tag{3}$$

where t_k is the hit time and m_k the local miss rate of level k, and the penalty of the last cache level is the main-memory latency. The textbook's worked example uses an L1 with 10% miss rate and 1-cycle latency, an L2 with 10% miss rate and 10-cycle latency, and a 100-cycle memory, giving $T = 1 + 0.1(10 + 0.1 \times 100) = 3$ cycles against 100 cycles for a flat memory [1, sec. 11.1.7, Example 147].

Equation (3) shows precisely what a larger last-level cache can and cannot do:

- it can lower m_{LLC}, so fewer accesses pay the main-memory latency;
- but if making the cache larger also lengthens its hit time t_{LLC}, part of the gain is lost on every LLC hit.

The design problem for a large LLC is therefore to add capacity while keeping the added hit latency small. That is the problem 3D stacking attacks (Part IV).

4.3 Classifying cache misses

The textbook divides misses into three classes [1, sec. 11.3.2]:

- **Compulsory (cold) misses** occur the first time a block is accessed.
- **Capacity misses** occur when the program's working set—the blocks it uses over a typical interval—is larger than the cache.
- **Conflict misses** occur in direct-mapped and set-associative caches when more blocks map to one set than the set can hold.

A larger LLC primarily removes capacity misses. It therefore helps workloads whose working set exceeds the original cache but fits in the larger one, and helps little when the working set either already fits or is far larger than either size. This simple argument predicts the strongly workload-dependent results examined in Part V.

4.4 Cache organisation and policies

Planned content (not yet written). Direct-mapped, set-associative and fully associative organisation; tags, sets and associativity; replacement policies (random, FIFO, LRU); write-back versus write-through [1, sec. 11.2]. Explain the dependence of hit time on cache size and associativity. About four pages; move any derivation over half a page to an appendix.

4.5 Beyond SRAM: alternative LLC technologies

Stacking SRAM is not the only way to obtain a larger last-level cache. Research has also explored denser memory technologies. Mittal and Vetter's survey notes that DRAM offers roughly eight times the density of SRAM, and that die-stacked DRAM can provide 8 to 15 times the bandwidth of off-chip DRAM at one half to one third of its latency, which has made stacked DRAM a candidate for very large LLCs [7, sec. 1]. Its longer access latency, the size of the tag store such a cache needs, and thermal management are the main design challenges the survey identifies [7, secs. 1–2]. Non-volatile memories such as STT-RAM offer high density and low leakage, but with much longer write latency and higher write energy than SRAM. Kim *et al.* therefore study hybrid LLCs that combine an SRAM region for write-intensive blocks with an STT-RAM region for the rest, and report a 26% average speedup in simulation from a better block-placement policy [8]. These approaches change the *technology* of the cache, whereas AMD's 3D V-Cache keeps SRAM and changes its *placement*; the comparison is taken up in Part VI.

5 Multicore Processors, Coherence and Interconnects

5.1 From one core to many

The textbook traces the move to multicore processors from the mid-2000s as transistor budgets continued to grow while single-core power and complexity limits tightened [1, sec. 12.1].

Multicore processors raise two questions that matter directly for chiplets: how cores keep a consistent view of shared data, and how requests travel between cores, caches and memory.

5.2 Cache coherence

When several caches can hold copies of the same address, cache coherence rules ensure that all threads observe writes to that address in a consistent order [1, sec. 12.4.2]. In a chiplet processor, a coherence transaction between cores on different compute dies must cross the die-to-die interconnect, so the physical partitioning becomes visible as latency (section 8).

Planned content (not yet written). Snooping versus directory protocols; MSI-style states at the level used by the textbook; why shared versus private LLC organisation matters for coherence traffic [1, ch. 12]. About four pages.

5.3 On-chip and on-package interconnection networks

Planned content (not yet written). Bus, ring, mesh and crossbar topologies; latency and bandwidth metrics; banked and non-uniform caches [1, ch. 12]. Bridge to Part III by asking what changes when a link leaves the die. About three pages.

Part III

Chiplet-Based Processor Architecture

Target length: about 25 pages. Draft status: yield model, CCD/IOD partitioning and IFOP characteristics drafted from AMD sources; cost modelling and scalability analysis pending.

6 Why Large Monolithic Dies Become Costly

6.1 The reticle limit and die size

A single die cannot be larger than the area a lithography tool can expose in one field. AMD's Hot Chips 33 presentation plots server CPU and GPU die sizes growing towards this reticle limit over roughly the last fifteen years [6]. For its second-generation EPYC ("Rome") processor, AMD states that the 64-core top configuration would not be practical as a monolithic chip and would lie well beyond the maximum reticle size [2].

6.2 Manufacturing yield

Manufacturing defects are scattered across a wafer, so a larger die is more likely to contain one. The simplest model assumes that defects fall independently with a fixed average density D_0, which gives a Poisson yield e^{-AD_0} for a die of area A. Real defect densities vary across a wafer and from wafer to wafer, and the Poisson model ignores this. This paper therefore uses the yield model adopted in the 3D-IC cost analysis of Dong and Xie [9, Eq. 15]:

$$Y_{\text{die}} = Y_{\text{wafer}} \cdot \frac{1 - e^{-2AD_0}}{2AD_0}, \tag{4}$$

where A is the die area, D_0 the average defect density and Y_{wafer} the wafer yield. This is one of the yield functions derived by Murphy, who averaged the Poisson yield over a defect density that varies from chip to chip; this form assumes the density is spread uniformly between 0 and $2D_0$ [10, Eq. 11]. Appendix C derives it and compares it with Murphy's preferred intermediate model.

Table 1 gives an illustrative calculation with $D_0 = 0.2$ defects/cm^2 and $Y_{\text{wafer}} = 1$. It is a model result, not a measured processor result.

Doubling the die area lowers the modelled yield from about 82.4% to 68.8% (figure 3). Cost per good die rises faster than area for two reasons: fewer dies fit on each wafer, and a smaller fraction

Table 1: Illustrative Murphy-model yield for $D_0 = 0.2$ defects/cm^2 and $Y_{\text{wafer}} = 1$

Die area	$2AD_0$	**Modelled yield Y_{die}**
100 mm^2 (1 cm^2)	0.4	82.4%
200 mm^2 (2 cm^2)	0.8	68.8%

of them work. The exact increase depends on wafer cost, process node, defect distribution and test cost; appendix C derives the cost-per-good-die expression and shows why partitioning a design into smaller dies can lower the silicon cost even after paying for extra interface area.

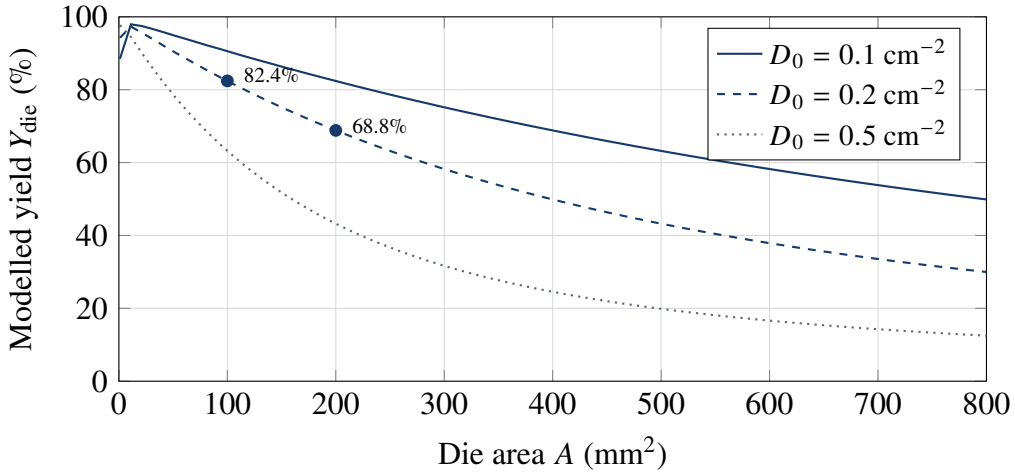


Figure 3: Murphy yield model (equation (4), $Y_{\text{wafer}} = 1$) for three illustrative defect densities. The marked points reproduce the worked example in section 6.2. Illustrative calculation, not measured processor data.

AMD's own material makes the same qualitative point. Its Zen 2 presentation lists "small die improve yield" among the motivations for the chiplet strategy [3], and its Hot Chips 33 presentation shows normalised cost per yielded square millimetre rising at each newer process node [6].

6.3 Uneven scaling of logic, SRAM and analog circuits

Moving a design to a newer node does not shrink every circuit equally. Naffziger *et al.* state that shrink factors at advanced nodes are much lower for analog circuitry than for digital logic and SRAM, and that memory interfaces and SerDes therefore gain very little performance, power or area from moving from 12 nm to 7 nm [2]. Building those circuits on the expensive leading-edge node wastes its cost. This observation is the basis of process-node specialisation (section 9). The related, more recent observation that SRAM is falling behind logic is taken up in section 11.

7 Compute Dies, I/O Dies and Core Complexes

AMD's Zen 2 products ("Rome" for servers and "Matisse" for desktops) are built from three kinds of die [2], [3]:

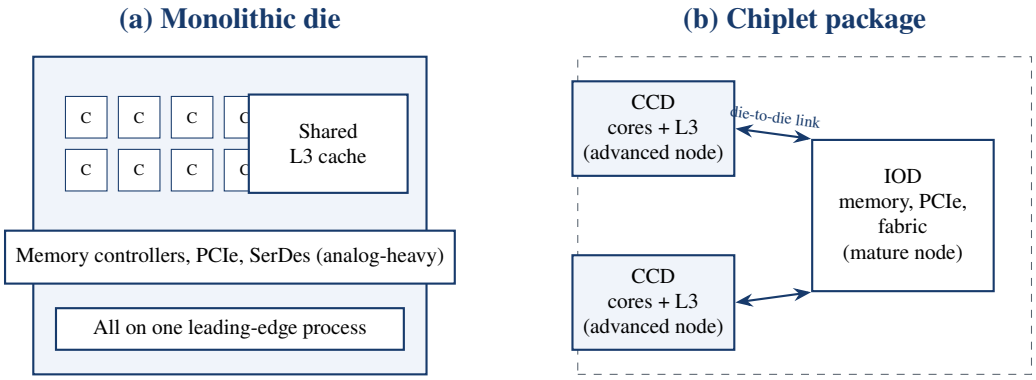


Figure 4: Schematic contrast between a monolithic processor die and a chiplet package with compute dies (CCDs) and an I/O die (IOD). Illustrative only; not to scale and not an actual AMD floorplan.

- an eight-core CCD (Core Complex Die) of about 74 mm^2 on a 7 nm process, containing the cores and their L3 cache;
- a server IOD (I/O Die) of about 416 mm^2 on a 12 nm process;
- a smaller client IOD of about 125 mm^2, which in a standalone package also serves as the X570 chipset.

The same CCD is reused across server and desktop products; only the IOD and the number of populated CCDs change [2]. Within each Zen 2 CCD, the cores are grouped into two four-core CCX (Core Complex)s, each with up to 16 MB of L3 [3].

The partition places latency-critical, logic-dense circuits—cores and caches—on the newest node, and places memory controllers, PCIe and other I/O, which scale poorly, on a cheaper mature node (figure 4).

Planned content (not yet written). Expand with the Rome and Matisse package organisation (CCD count, IOD placement) from [2] Figs. 2.2.1–2.2.4, and compare with first-generation EPYC ("Naples"), whose four dies each carried their own memory and I/O. Avoid reverse-engineering undisclosed details.

8 Infinity Fabric and Die-to-Die Communication

Splitting a processor into dies turns some on-chip wires into package-level links. AMD connects each CCD to the IOD with an IFOP (Infinity Fabric On-Package) serial link routed over 10–20 mm of organic package [2]. AMD reports, for Rome [2]:

- about 55 GB/s of effective bandwidth per IFOP link at a 1.46 GHz fabric clock;
- a round-trip latency from CCD to IOD and back of 13 fabric clocks, under 9 ns;
- longer reach than a 2.5D silicon interposer, which allowed two-deep columns of chiplets and hence eight CCDs in one package;
- a 19% average reduction in memory latency for the socket compared with the previous generation, attributed to the central IOD, decoupled clocks and the latency-optimised link.

The predecessor "Zeppelin" link achieved about 2 pJ/bit [2]. These are AMD-reported figures for specific configurations and are not independent measurements.

8.1 Latency, bandwidth and interconnect power

Planned content (not yet written). Interpret the figures above with the textbook models: a die-to-die hop adds latency to every L3 miss that goes to memory (raising $\text{penalty}_{\text{LLC}}$ in equation (3)); link energy per bit multiplies into interconnect power (equation (2)); per-link bandwidth caps the memory bandwidth available to one CCD. Discuss cross-CCD coherence traffic and NUMA effects, using [5] for the Zen 3 EPYC NUMA options. About five pages.

8.2 Scalability

Planned content (not yet written). How the number of CCDs, IOD links and memory channels scale together; diminishing returns and bandwidth balance (AMD notes that four or more CCDs are populated in most Rome configurations to balance bandwidth [2]). About three pages.

9 Yield, Process-Node Specialisation and Cost

Chiplets combine two cost advantages. First, small dies yield better (section 6.2). Second, each die can use the process that suits it: AMD keeps memory interfaces and SerDes on 12 nm because they would see little benefit at 7 nm [2]. AMD also tests every CCD before assembly and can down-bin a CCD damaged during assembly, placing it in a known low-power state [2], which limits the cost of combining several dies in one package.

These benefits are not free. AMD describes the design as requiring package–silicon co-design, new interconnect circuits, a mixed-process package (with copper pillars on the 12 nm IOD to match die heights), and new test and power-management infrastructure [2].

Planned content (not yet written). Quantitative cost comparison of monolithic and chiplet designs using the model in appendix C and the 3D-IC cost model of Dong and Xie, including their result that splitting pays off only above a size threshold and their die-to-wafer versus wafer-to-wafer comparison [9]; discussion of packaging and test cost.

10 Summary of Part III

Planned content (not yet written). Apply the three-part test: *bottleneck addressed* (die size, yield, uneven scaling); *benefit* (core-count scaling, reuse across markets, cost); *new trade-offs* (die-to-die latency and power, packaging complexity, NUMA-like behaviour). Close by noting that each CCD still carries its own planar L3, which motivates Part IV.

Part IV

3D-Stacked Cache and AMD 3D V-Cache

Target length: about 25 pages. Draft status: argument for stacking, planar limits, side-by-side versus vertical comparison and hybrid-bonding facts drafted; thermal and workload chapters need independent sources.

11 Why Cache Capacity Matters

Sections 4.2 and 4.3 showed that a larger LLC lowers the miss rate by removing capacity misses, provided its hit time does not grow too much. Whether that matters depends on the workload: a program whose working set already fits in the cache gains little, while one whose working set lies just above the cache size can gain a great deal.

AMD's ISSCC 2022 presentation of 3D V-Cache opens with two charts that frame this part [4, slides 5–6]. The first, titled "Large L3 Caches Provide IPC Uplift", plots IPC uplift on a linear scale rising steadily as L3 size grows from 1× to 32×; the slide gives no values on the uplift axis and does not name the workloads. The second, "Silicon Area Scaling by Function", shows logic area continuing to shrink from 28 nm to 5 nm while SRAM scaling falls behind from about 7 nm and analog circuits shrink least; a companion chart on the same slide shows normalised cost per yielded mm^2 (for a 250 mm^2 die) rising from 45 nm to 5 nm. The slide summarises these as "SRAM scaling & cost = barriers to large on-die caches". This extends Naffziger *et al.*'s 2020 observation that analog circuits scale worst [2]: by the 5 nm generation SRAM, too, has become comparatively expensive.

Both charts are vendor material, and the first has no stated workloads or values; neither is evidence that every program benefits. Part V tests the claim against published measurements.

12 Why Expanding Cache in Two Dimensions Became Difficult

Growing the L3 in the plane of the compute die meets four related obstacles.

12.1 Area and yield

SRAM cells no longer shrink at the rate of logic (section 11), so a cache of a given capacity occupies a growing share of the die at each new node. AMD lists cost, product flexibility and

latency as the resulting barriers to large on-die caches [4, slide 6]. Mittal and Vetter add that, if the large caches needed by future workloads were built from SRAM, they could occupy up to 90% of the chip area [7, sec. 1]. A larger die yields fewer good dies per wafer (section 6.2), so doubling the L3 on a planar CCD would raise the cost of every CCD, including those sold for workloads that do not need the extra cache. It would also make the CCD less flexible as a building block reused across products.

12.2 Wire length and access latency

A larger planar cache spreads over a larger area, so the average distance between a core and the cache bank holding its data increases. Longer wires have more resistance and capacitance; their RC delay increases access latency, and the energy needed to charge their capacitance increases dynamic power (equation (2)). In terms of equation (3), part of the miss-rate gain from a bigger cache is paid back as a longer hit time t_{LLC}. AMD makes the same argument for stacking: compared with spreading a large cache across the plane, 3D integration "improves effective memory latency" and "reduces long datapath and I/O's dynamic powers" [4, slide 7].

12.3 Power

Large SRAM caches consume dynamic power on every read and write and leakage power even when idle (section 3.2). Mittal and Vetter's survey of DRAM-cache architectures argues that the low density and high leakage power of SRAM make very large SRAM caches infeasible, and that such caches may need expensive cooling, such as liquid cooling, which adds substantial cost and complexity [7, sec. 1]. This is one reason researchers have explored denser technologies, such as stacked DRAM, for last-level caches.

12.4 Summary of the planar limits

Planar expansion therefore trades miss rate against die cost, hit latency and power all at once. The question for the next section is whether a separate cache die can break that coupling.

13 Side-by-Side Versus Vertically Stacked Cache Dies

If extra cache is to be made on its own die, it can be placed either *beside* the compute die in the package or *on top of* it (figure 5). Both are valid chiplet designs; the choice is a trade-off rather than a matter of one being impossible.

13.1 Placing the cache die beside the compute die

A side-by-side cache die is a perfectly workable chiplet design, but it has three costs:

- **Package footprint.** The extra die occupies additional area next to the CCD. This enlarges the *package*, even though it does not enlarge the compute die itself. The distinction between

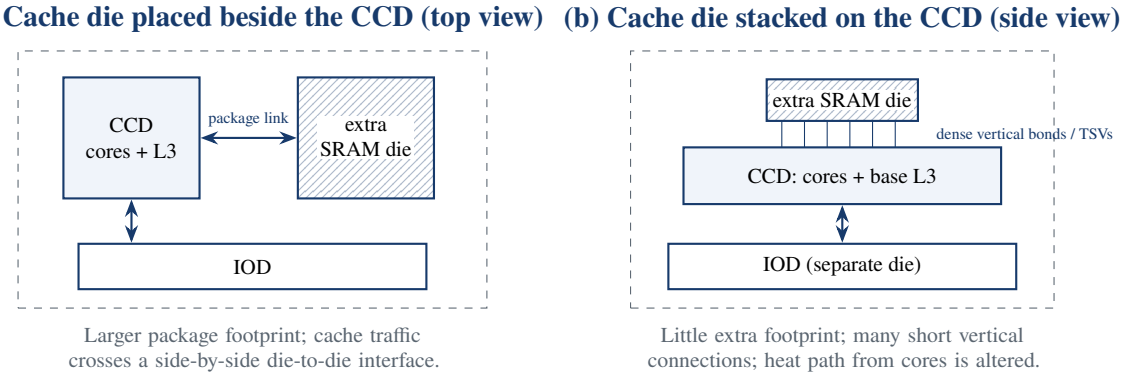


Figure 5: Two ways to add cache capacity to a compute die. Illustrative arrangements only; not actual AMD package layouts and not to scale.

die area (one piece of silicon) and *package footprint* (the area of the assembled processor) is important throughout this part.

- **Interconnect.** Every access to the extra cache must cross a die-to-die interface through the package or an interposer. Package-level connections are far coarser than on-die wiring, which limits how many parallel connections—and hence how much bandwidth—the link can offer.
- **Latency and power.** Data must travel between the dies, adding interface circuitry and wire delay to every access to the extra cache, and energy to every bit moved.

A side-by-side die does not automatically push the CCD farther from the IOD: the package designer chooses where each die goes, and the cache die could be placed on the far side of the CCD. However, package area is limited, the CCD–IOD links are already routed with care to minimise memory latency [2], and every additional die competes for the same routing resources. At high signalling rates, longer and more numerous package routes bring RC delay, power, signal-integrity and routing-congestion problems. These can be mitigated with better interconnect design, buffering and packaging technology, so they are engineering costs rather than impossibilities.

The more important comparison is therefore not the CCD-to-IOD distance but the latency and bandwidth of the path *between the cores and the additional cache*.

AMD's own framing of the choice matches this analysis. Its ISSCC 2022 presentation illustrates a hypothetical processor whose large cache is split off onto a side-by-side chiplet, and states that "2.5D chiplets can provide product flexibility and reduce cost" but that "3D can be even better", because it improves effective memory latency, reduces long-datapath and I/O dynamic power, and "fits more transistors within a given package cavity size" [4, slide 7].

13.2 Stacking the cache die on the compute die

Vertical stacking changes the trade-off:

- **Footprint.** The cache die overlaps the CCD's existing footprint, so capacity grows with little additional package area.

- **Interconnect density.** The two dies can be joined by a very large number of short vertical connections, which supports the bandwidth a large cache needs and keeps the added distance small.
- **Thermal design.** The stack can obstruct heat flow from the compute die, depending on which die is on top and what lies above the cores. This is the central new engineering problem (section 16).

Table 2: Qualitative comparison of side-by-side and vertically stacked cache dies

Factor	Cache die beside the CCD	Cache die stacked on the CCD
Package footprint	Larger	More compact
Connections	Side-by-side package or interposer link	Dense vertical bonds and through-silicon vias
Cache access latency	Depends on the interface design	Can benefit from short, dense connections
Cooling	Easier to arrange	More difficult; depends on stack order
Manufacturing	Extra die plus package assembly	Extra die plus bonding, thinning and stacking

Table 2 summarises the comparison. The case for vertical stacking is not that side-by-side chiplets cannot work. It is that stacking can add substantial cache capacity without enlarging the package by the same amount, with potentially better interconnect characteristics. The question worth investigating is *when the extra complexity of 3D integration is worth the smaller footprint and denser interconnect*.

14 Stacked-Die Integration and Hybrid Bonding

14.1 The first-generation 3D V-Cache stack

AMD's Hot Chips 33 presentation describes the first-generation stack as a 64 MB L3 cache die attached to an up-to-eight-core Zen 3 CCD by a direct copper-to-copper bond, with TSV (through-silicon via)s carrying signals and power through the CCD [6]. AMD's EPYC 7003 documentation states that the L3 cache tiles are stacked on the central L3 portion of the base CCX [5]. The ISSCC 2022 presentation describes the three components of the stack and how they are assembled [4, slides 10–12, 14, 16–17]:

- the Zen 3 CCD, built on TSMC's 7 nm process with eight cores sharing 32 MB of L3 in an area of 81 mm^2, with V-Cache support designed in "from Day 1";
- the extended L3 die (L3D), also 7 nm, carrying 64 MB of L3 in 41 mm^2, roughly half the CCD area;
- structural silicon dies, which provide mechanical support for the thinned CCD and a path for heat from the CPU cores.

The CCD is mounted face-down on the package substrate through conventional C4 bumps and connects to the L3D through TSVs. The L3D is mounted face-down on the CCD by hybrid

bonding, using TSMC's SoIC process with a minimum TSV pitch of 9 µm, while the structural dies are oxide-bonded beside it. Support silicon brings the stack to the same height as an unstacked CCD, so server parts with and without V-Cache fit the same package. The cache die sits over the CCD's central L3 region [5] and the structural silicon over the cores, so the hottest circuits are covered only by passive silicon.

14.2 Hybrid bonding versus micro-bumps

Earlier 3D stacking typically joined dies with solder micro-bumps. Hybrid bonding instead bonds aligned copper pads and surrounding dielectric directly. AMD reports, from its internal engineering analysis [6]:

- a hybrid-bond pitch of 9 µm, compared with 50 to 36 µm for micro-bump 3D;
- more than 15× the interconnect density of micro-bump 3D;
- more than 3× better interconnect power efficiency than micro-bump 3D;
- lower TSV capacitance and inductance, which benefits signal and power delivery.

AMD's EPYC 7003 documentation similarly claims over 200× the interconnect density of 2D packaging and over 15× that of solder-bump 3D [5]. These are AMD's own comparisons and should be read as engineering claims.

Planned content (not yet written). Explain why interconnect density matters architecturally (parallel connections → bandwidth; short connections → low latency and energy), and how this addresses the side-by-side costs in section 13. Keep fabrication detail at the architectural level.

15 Cache Access, Latency, Bandwidth and Hierarchy

With the stacked die, the L3 of a Zen 3 CCD grows from 32 MB to 96 MB, shared by up to eight cores [4], [5]; an EPYC 7003 processor with eight such CCDs carries up to 768 MB of L3 [5]. AMD reports up to 2 TB/s of bandwidth for the V-Cache [4]. The extra capacity remains part of the same L3 level, so the cache hierarchy above it (private L1 and L2 per core) is unchanged [5].

AMD's presentation gives the cost of the extra capacity in latency: the 96 MB L3 remains 16-way set associative with a 32 B/cycle interface to each core, delivers more than 2 TB/s of L3 bandwidth, and adds *four cycles* of latency [4, slide 20]. Each die's portion of the L3 has its own data, tag and LRU arrays, so a tag lookup does not have to cross between dies before the data array is accessed [4, slide 20].

Equation (3) shows how to judge this trade-off. For an access that has already missed in L2, the extra capacity changes the average cost by approximately

$$\Delta T \approx \Delta t_{\mathrm{L3}} - \Delta m_{\mathrm{L3}} \cdot t_{\mathrm{mem}}, \tag{5}$$

where $\Delta t_{\mathrm{L3}} = 4$ cycles is the added hit latency, Δm_{L3} the reduction in the L3 miss rate and t_{mem} the main-memory latency in cycles. As a purely illustrative example, if main memory took 300 cycles, the larger L3 would pay off once it removed more than about $4/300 \approx 1.3$ percentage points of L3 misses. A workload whose working set already fits in 32 MB gains nothing and

pays the four cycles on every L3 hit; a workload whose working set lies between 32 MB and 96 MB can gain substantially. **[TODO:** Replace the illustrative 300-cycle memory latency with a cited value, or keep it explicitly labelled as hypothetical.**]**

The L3 remains private to each CCD, so a thread running on one CCD does not benefit from another CCD's stacked cache.

16 Thermal Management, Power Delivery and Manufacturing Complexity

Stacking changes the heat path described in section 3.2: heat from the cores must now pass through additional silicon and bonding layers before it reaches the heat spreader. The first-generation design limits the penalty by placing the cache die over the CCD's central L3 region [5] and covering the cores only with structural silicon, whose stated purposes include "thermal dissipation for CPU cores" [4, slide 12]. AMD also credits hybrid bonding with "superior thermal conductance" compared with micro-bump stacking [4, slide 15], although this is an AMD claim based on internal simulation. Earlier academic work points the same way: in a study reviewed by Mittal and Vetter, Black *et al.* stacked an 8 MB SRAM cache directly on a dual-core processor and found that none of their stacked-cache designs changed the thermal profile significantly, although stacked SRAM raised temperature more than stacked DRAM because of its higher power density [7, p. 1853].

Power must also be delivered upwards. The L3D receives its supplies through power TSVs placed in channels between the CCD's array macros, and its arrays use separate rails for the bit cells and the peripheral circuits [4, slides 24–25].

Planned content (not yet written). Die thinning, bonding and test complexity (design-for-test is listed among the topics of [4]); frequency and voltage limits of stacked-cache parts. **[TODO:** Identify independent sources on thermal behaviour and on frequency or voltage limits of stacked-cache parts.**]** **[VERIFY:** Later generations are reported to place the cache die beneath the CCD; confirm from a primary source before mentioning.**]**

17 Workloads That Benefit From a Larger LLC

AMD reports a 15% average gaming performance uplift from V-Cache [4]. Its Hot Chips 33 comparison of a 12-core 3D-chiplet prototype with a Ryzen 9 5900X at the same fixed frequency reports an average gain of 15% across a small set of games [6]. Holding frequency fixed is valuable because it isolates the cache effect, but the result covers only the disclosed games and configuration and should not be generalised.

The ISSCC 2022 presentation adds product results, again from AMD's own testing [4, slides 29, 31, 35]. The eight-core Ryzen 7 5800X3D is reported as about 15% faster in games than the twelve-core Ryzen 9 5900X at 1080p with high settings, ranging from up to 1.36× in one title to a tie in another. On servers, a 16-core EPYC with V-Cache completed about 66% more electronic-design (RTL simulation) jobs per hour than a 16-core EPYC 73F3 without it. The spread from a tie to 1.36× already illustrates the workload dependence predicted in section 4.3;

section 24 discusses what else differs between the compared parts.

Planned content (not yet written). Characterise cache-sensitive workloads in terms of working-set size and reuse (section 4.3): games, some simulation and electronic-design workloads, databases. Characterise insensitive workloads (small working sets; streaming data with little reuse). All quantitative claims to come from Part V.

18 Summary of Part IV

Planned content (not yet written). Three-part test: *bottleneck* (planar cache cost, latency and power); *benefit* (tripled L3 per CCD with dense, short connections); *new trade-offs* (heat removal, bonding and test complexity, product cost, workload dependence).

Part V

AMD Case Study and Quantitative Analysis

Target length: about 25 pages. Draft status: generational progression and method drafted; benchmark data collection not started.

19 From Zen 2 Chiplets to Zen 3 and 3D V-Cache

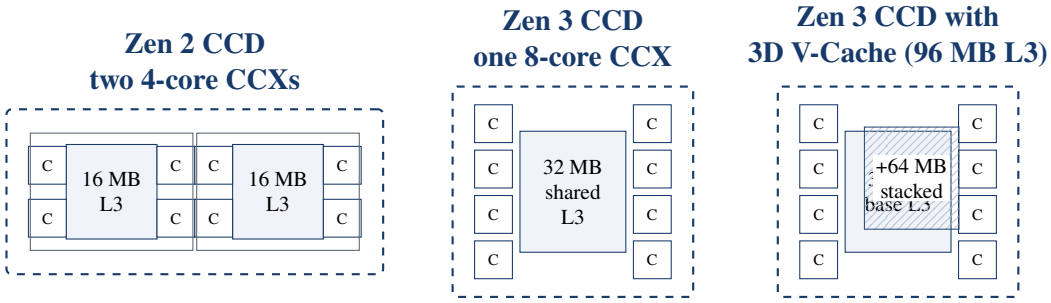


Figure 6: Schematic progression of L3 organisation per compute die from Zen 2 to Zen 3 and Zen 3 with 3D V-Cache, based on AMD's descriptions [3], [4], [5]. Logical organisation only; not a floorplan.

Table 3 and figure 6 summarise the changes in cache organisation that matter for this paper, using only AMD's published descriptions.

Two steps stand out. Zen 3 did not add L3 capacity per CCD but changed its *sharing*: each core can now use the whole 32 MB rather than only its own CCX's 16 MB [3], [5]. 3D V-Cache then tripled the capacity seen by each core without changing the CCD's footprint or the sharing structure [4], [5]. These two steps give a natural experiment for the effect of L3 capacity, as long as other differences are controlled (section 24).

Planned content (not yet written). Extend the progression to later Zen generations with X3D variants using primary AMD sources only. **[VERIFY:** collect generation-by-generation L3 sizes, stack orientation and frequency limits from AMD documentation before writing.**]**

Table 3: L3 organisation per compute die across the generations studied (AMD-published figures)

	Zen 2 CCD	**Zen 3 CCD**	**Zen 3 CCD with 3D V-Cache**
Cores per CCD	8	8	8
CCX organisation	Two 4-core CCXs	One 8-core CCX	One 8-core CCX
L3 shared by one core	Up to 16 MB (its CCX)	32 MB	96 MB
L3 per CCD	2×16 MB	32 MB	32 MB base + 64 MB stacked
Private L2 per core	512 KB	512 KB	512 KB
CCD area	74 mm^2	81 mm^2	81 mm^2 + 41 mm^2 L3D
Added L3 latency	—	—	+4 cycles
Sources	[2], [3]	[4], [5]	[4], [5]

20 Comparing Architectural Configurations

Planned content (not yet written). Table of configuration pairs that differ mainly in L3 capacity (for example, a Zen 3 desktop or server part and its V-Cache counterpart): core count, base and boost clocks, power limit, memory support, L3 per CCD. **[TODO:** All figures from AMD product documentation, with access dates.]

21 Method for Analysing Published Evidence

The quantitative analysis uses published research and benchmarks only (figure 7). This is the most manageable option given that the group does not have access to the relevant hardware. Its limitation is dependence on the experimental conditions and measurements that others chose to publish; the analysis can only be as controlled as its sources.

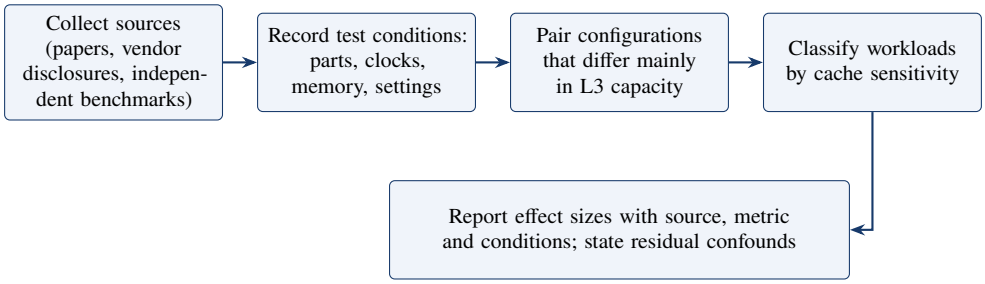


Figure 7: Planned evidence-analysis procedure for Part V (published research and benchmarks only; no new hardware measurement).

Each source is recorded with: the processors compared; clock frequencies (fixed or boost); memory type, speed and timings; power limits; software and settings; the metric (for example, frames per second, IPC, runtime or throughput); and whether the source is the vendor or an independent party. Results are reported as relative changes with the source, metric and conditions stated, never as a single headline number.

Planned content (not yet written). Source-selection criteria (peer-reviewed or methodologically transparent benchmark publications; vendor data used only when labelled as such); handling of conflicting results; how averages across workloads are formed (geometric mean of ratios).

22 Published Benchmarks and Performance Measurements

The evidence currently in hand is entirely AMD-reported (table 4). Independent measurements are still to be collected.

Table 4: AMD-reported performance figures relevant to this study (vendor claims, not independent measurements)

Claim	Conditions stated	Source
About 15% faster gaming, 8-core Ryzen 7 5800X3D vs 12-core Ryzen 9 5900X; per-title results from a tie to 1.36×	Six games at 1080p, high preset; both with DDR4-3600 and RTX 3080; AMD testing, Dec. 2021	[4, slides 29, 35]
About 66% more RTL-simulation jobs per hour (24.4 to 40.6), 16-core EPYC with V-Cache vs 16-core EPYC 73F3	Synopsys VCS; same reference platform; AMD testing, Sept. 2021	[4, slides 31, 35]
19% average IPC gain, Zen 3 vs Zen 2	25 workloads at a locked 4 GHz, 8-core desktop parts	[4, slides 10, 35]
15% average gaming uplift, 12-core 3D-chiplet prototype vs Ryzen 9 5900X	Both at the same fixed frequency; disclosed games only	[6]
15% IPC improvement from Zen to Zen 2	Many simultaneous changes, including doubled L3	[3]
19% average memory-latency reduction, Rome vs Naples	Central IOD, decoupled clocks, optimised IFOP	[2]

[TODO: Collect independent benchmark studies covering gaming, technical computing, databases and general productivity for paired V-Cache and non-V-Cache parts; add each to Zotero with a reading note before citing.]

23 Cache-Sensitive Versus Cache-Insensitive Workloads

Planned content (not yet written). Classify workloads from the collected evidence by the size of the stacked-cache benefit, and explain the pattern with working-set size and the miss classes of section 4.3. Include cases where the V-Cache part is slower (for example, where lower clocks outweigh a cache that the workload does not need), if the evidence shows them.

24 Separating Cache Effects From Other Factors

A difference in measured performance between two processors can rarely be attributed to cache size alone. Using equation (1) and equation (3), the main confounding factors are:

- **Clock frequency.** If the larger-cache part runs at a different frequency, f in equation (1) changes as well as IPC. Fixed-frequency comparisons, like the one AMD reports for its prototype [6], remove this factor.
- **Core count and topology.** More cores or a different number of CCDs changes throughput and the cache available per core. AMD's headline desktop comparison sets an eight-core, single-CCD Ryzen 7 5800X3D against a twelve-core, two-CCD Ryzen 9 5900X [4, slide 29], so it mixes cache capacity with core count, CCD topology and clock speed. Its server comparison holds the core count at 16 but uses the EPYC 73F3 as the baseline, an F-series model that AMD positions for high frequency [4], [5].
- **Memory bandwidth and latency.** Memory speed, timings and the fabric clock change the miss penalty in equation (3) and can make a larger cache appear more or less valuable.
- **Concurrent microarchitectural changes.** Generational comparisons mix many changes. AMD's 15% Zen-to-Zen 2 IPC figure, for example, combines a new branch predictor, a larger op cache, wider floating-point paths and a doubled L3, among others [3]; it cannot be credited to the cache.
- **Power limits and software.** Package power limits, operating-system scheduling across CCDs, and application versions all affect results.

Planned content (not yet written). For each collected comparison, record which of these factors are controlled and which remain, and grade the strength of the cache-attribution accordingly.

25 Summary of Part V

Planned content (not yet written). Answer RQ4 with the collected evidence: where the stacked L3 helps, by how much, under which conditions, and with what confidence.

Part VI

Comparative Synthesis, Limitations and Future Directions

Target length: about 20 pages. Draft status: comparison framework and future-directions outline drafted; conclusions await Parts IV–V.

26 Monolithic, Chiplet-Based and Vertically Stacked Designs Compared

Table 5 compares the three organisations against the bottleneck–benefit–trade-off test of section 1.3. Entries are qualitative and will be revised against the evidence gathered in Part V.

Table 5: Qualitative comparison of the three design approaches (provisional)

	Monolithic die	**Chiplets (2D package)**	**Chiplets with stacked cache**
Main limit addressed	—	Die size, yield, uneven node scaling	Planar cost and wire length of a large LLC
Communication	All on-die	Die-to-die links through the package	Adds dense vertical links between cache and compute dies
Memory latency	Shortest on-die paths	Extra die-to-die hop to memory	Fewer trips to memory for cache-sensitive work
Cost and yield	Falls steeply with die area	Smaller dies, mixed nodes, tested before assembly	Extra die and bonding step; capacity only where sold
Thermal	One heat source per die	Heat spread over several dies	Stack alters heat path from cores
New trade-off	Scaling ceiling	Latency, link power, packaging	Heat, test complexity, workload dependence

27 Interactions Between Chiplet Organisation and Cache Design

Planned content (not yet written). Develop the argument that chiplets and stacked cache are complementary: the CCD's small, self-contained die with its own L3 makes it a natural base for a stacked cache, and stacking lets a product line offer large-cache variants without changing the base CCD or the package layout. Discuss the per-CCD L3 boundary: stacked capacity helps threads on that CCD only, which interacts with scheduling and with cross-CCD communication (section 8).

28 Workload-Dependent Trade-Offs and Remaining Bottlenecks

Planned content (not yet written). Synthesise Parts III–V: which workloads favour more cores, which favour more cache, which favour frequency; the remaining bottlenecks (memory bandwidth, die-to-die latency, heat removal from stacked dies).

29 Future Directions in Heterogeneous Integration and Advanced Packaging

AMD's Hot Chips 33 presentation sketches integration schemes that go beyond stacking cache on cores, including memory stacked on compute, cores stacked on cores or on uncore logic, and splitting or "folding" circuits across dies; AMD presents these as schemes that advanced packaging could enable but monolithic design could not [6]. These are vendor projections, not products.

Planned content (not yet written). Discuss the architectural questions these raise (heat density of logic-on-logic stacks, testability, design tools, standard die-to-die interfaces). Candidate sources to be identified and read; no claims about specific future products without primary sources.

30 Limitations of This Study

- The quantitative analysis relies on published measurements and is limited by their experimental conditions and by the availability of comparable data.
- Much of the detailed technical information about AMD designs comes from AMD itself; such claims are attributed to AMD and treated as engineering claims rather than independent results.
- The study works at the architectural level and does not model physical design, packaging processes or thermal behaviour in detail.
- **[TODO:** Add limitations arising from the final evidence set.]

31 Conclusions

Planned content (not yet written). Answer RQ1–RQ4 in turn. Write only after Parts IV and V are complete.

32 Open Research Questions

- When is the extra complexity of 3D integration worth its reduction in footprint and its denser interconnect, compared with a side-by-side cache chiplet?
- How should operating systems and runtimes place threads and data when L3 capacity differs between compute dies?
- What limits the stacking of logic on logic, as opposed to SRAM on logic, from a thermal and power-delivery point of view?
- **[TODO:** Refine after the evidence review.]

References

[1] S. R. Sarangi, *Basic Computer Architecture*, Version 3.09. Author's online edition, 2025.

[2] S. Naffziger, K. Lepak, M. Paraschou, and M. Subramony, "AMD chiplet architecture for high-performance server and desktop products," in *2020 IEEE International Solid-State Circuits Conference (ISSCC)*, IEEE, 2020, pp. 44–45. DOI: 10.1109/ISSCC19947.2020.9063103

[3] D. Suggs, D. Bouvier, M. Clark, K. Lepak, and M. Subramony, "AMD Zen 2," in *2019 IEEE Hot Chips 31 Symposium (HCS)*, IEEE, 2019.

[4] J. Wuu et al., "3D v-cache: The implementation of a hybrid-bonded 64MB stacked cache for a 7nm x86-64 CPU," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, IEEE, 2022, pp. 428–429. DOI: 10.1109/ISSCC42614.2022.9731565

[5] Advanced Micro Devices, Inc., "AMD EPYC 7003 processor microarchitecture overview," Advanced Micro Devices, Inc., 2022, 57075. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/white-papers/overview-amd-epyc7003-series-processors-microarchitecture.pdf

[6] R. Swaminathan, "Advanced Packaging: Enabling Moore's Law's Next Frontier Through Heterogeneous Integration," in *2021 IEEE Hot Chips 33 Symposium (HCS)*, Virtual: IEEE, 2021. [Online]. Available: https://hc33.hotchips.org/assets/program/tutorials/2021%20Hot%20Chips%20AMD%20Advanced%20Packaging%20Swaminathan%20Final%20%2020210820.pdf

[7] S. Mittal and J. S. Vetter, "A Survey Of Techniques for Architecting DRAM Caches," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 6, pp. 1852–1863, Jun. 1, 2016, ISSN: 1045-9219. DOI: 10.1109/TPDS.2015.2461155 Accessed: Oct. 28, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/7181712/

[8] B. Kim, Y. Kim, P. Nair, and S. Hong, "Exploiting Data Compression for Adaptive Block Placement in Hybrid Caches," *Electronics*, vol. 11, no. 2, p. 240, Jan. 12, 2022, ISSN: 2079-9292. DOI: 10.3390/electronics11020240 Accessed: Oct. 28, 2026. [Online]. Available: https://www.mdpi.com/2079-9292/11/2/240

[9] X. Dong and Y. Xie, "System-level cost analysis and design exploration for three-dimensional integrated circuits (3D ICs)," in *2009 Asia and South Pacific Design Automation Conference*, Yokohama, Japan: IEEE, Jan. 2009, pp. 234–241, ISBN: 978-1-4244-2748-2. DOI: 10.1109/ASPDAC.2009.4796486 Accessed: Oct. 28, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/4796486/

[10] B. Murphy, "Cost-size optima of monolithic integrated circuits," *Proceedings of the IEEE*, vol. 52, no. 12, pp. 1537–1545, 1964, ISSN: 0018-9219. DOI: 10.1109/PROC.1964.3442 Accessed: Oct. 28, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/1445372

A Plagiarism Report

[TODO: No plagiarism-check evidence has been supplied for this draft. Each version needs its own evidence; Draft 1's report does not apply. Add the tool's displayed result for this draft and describe it only as what the tool displayed.]

B Reading Indices

[TODO: No readability indices have been supplied for this draft. Add the values exactly as displayed by the analysis tool. Readability indices estimate linguistic complexity only; they do not measure technical correctness or the strength of the argument.**]**

C Yield and Cost-per-Good-Die Model

This appendix derives the illustrative cost model used in section 6.2. It is a textbook-style model for reasoning about trends, not a model of any real AMD product or foundry process. The yield expressions follow Murphy [10] and Dong and Xie [9]; the numerical examples are this paper's own.

C.1 From Poisson to Murphy yield

If yield-killing defects fall independently and uniformly with a *fixed* density D per unit area, the number of defects on a die of area A is Poisson-distributed with mean AD. A die is good only if it has no defects, so its yield is

$$Y_{\text{Poisson}}(A, D) = e^{-AD}.$$

In practice the defect density is not fixed: it varies from chip to chip and from slice to slice. Murphy therefore averaged the Poisson yield over a normalised distribution $f(D)$ of defect densities, $Y = \int_0^\infty e^{-DA} f(D)\, dD$, and solved it for three simplified distributions [10, Eqs. 7–11]. If D is spread uniformly (a "rectangular" distribution) between 0 and $2D_0$, so that its mean is D_0, then

$$Y_{\text{Murphy}}(A) = \frac{1}{2D_0} \int_0^{2D_0} e^{-AD}\, dD = \frac{1 - e^{-2AD_0}}{2AD_0}.$$

This is Murphy's Eq. 11, and multiplying it by the fraction of wafers that survive processing, Y_{wafer}, gives the expression Dong and Xie use (their Eq. 15), which is equation (4).

Murphy also considered a narrower, "triangular" distribution, which gives

$$Y_{\text{triangular}}(A) = \left(\frac{1 - e^{-AD_0}}{AD_0} \right)^2.$$

He regarded the zero-width (Poisson) and rectangular cases as extremes that are "almost certainly inappropriate" and used the triangular form for most of his results, while noting that the three forms differ little at high yields [10, p. 1539]. This paper keeps the rectangular form because it is the one used in the 3D-IC cost model of Dong and Xie; at the yields in the worked example the choice changes the result by less than one percentage point. Because the averaged expression is never smaller than e^{-AD_0} for the same mean density, Murphy's model is less pessimistic than the Poisson model for large dies.

With $D_0 = 0.2\,\text{cm}^{-2}$ and $Y_{\text{wafer}} = 1$: for $A = 1\,\text{cm}^2$, $Y = (1 - e^{-0.4})/0.4 \approx 0.824$; for $A = 2\,\text{cm}^2$, $Y = (1 - e^{-0.8})/0.8 \approx 0.688$. For comparison, the triangular model gives 0.821 and 0.679, and the Poisson model 0.819 and 0.670.

C.2 Cost per good die

Let C_w be the cost of a processed wafer and $N(A)$ the number of whole dies of area A that fit on it. Ignoring edge losses, $N(A) \approx W/A$ for usable wafer area W. Dong and Xie use a more exact dies-per-wafer expression that also counts the partial dies lost at the wafer edge [9]. The cost per

good die is

$$C_{\text{good}}(A) = \frac{C_w}{N(A)\,Y(A)} \approx \frac{C_w}{W} \cdot \frac{A}{Y(A)}.$$

Doubling the area therefore multiplies the cost per good die by approximately $2\,Y(A)/Y(2A)$. In the example above (from 1 to $2\,\text{cm}^2$ with $D_0 = 0.2\,\text{cm}^{-2}$), the factor is $2 \times 0.824/0.688 \approx 2.40$: the larger die costs about 2.4 times as much, not twice as much.

C.3 Splitting a design into chiplets

Suppose a design of area A is split into k chiplets of area A/k, each with an additional interface overhead δ. If each chiplet is tested before assembly, so that only known-good dies are packaged, the silicon cost of the assembled part is approximately

$$k \cdot C_{\text{good}}\left(\tfrac{A}{k} + \delta\right) \approx \frac{C_w}{W} \cdot \frac{A + k\delta}{Y\left(\tfrac{A}{k} + \delta\right)}.$$

Smaller chiplets yield better, so the denominator grows with k, while the overhead term $k\delta$ in the numerator also grows. As an illustration with $D_0 = 0.2\,\text{cm}^{-2}$, a $4\,\text{cm}^2$ design split into four chiplets of $1\,\text{cm}^2$ plus a $0.05\,\text{cm}^2$ interface each costs about $(4.2/0.817)/(4/0.499) \approx 0.64$ of the monolithic silicon, a saving of roughly 36%. The saving grows for larger designs and higher defect densities. Dong and Xie reach the same qualitative conclusion for die stacking: it becomes cheaper than a single planar die only above a size threshold, and testing each die before stacking (die-to-wafer bonding) avoids multiplying together the yields of all dies in the stack [9]. The model omits packaging, assembly yield, test cost and the use of different process nodes for different chiplets, all of which section 9 discusses qualitatively.

Index

AMAT, 18

cache miss, 19

dynamic power, 17

hybrid bonding, 29

Infinity Fabric, 23

leakage power, 17

memory hierarchy, 18
Moore's law, 12
Murphy yield model, 21

performance equation, 16
plagiarism report, 40

readability indices, 41
reticle limit, 21

yield, 21, 42